

Professional Coding Specialist

COS Pro 파이썬 2 급

4 강-9 강. 모의고사 1 차

모의고사 1 차 1-10 번 정답

과정 소개

모의고사(YBM 샘플) 문제를 통해 실전을 대비할 수 있다. (회차당 10 문항, 각 문항별 문제 풀이)

학습 목차

모의고사 1 차 문제 1 번
모의고사 1 차 문제 2 번
모의고사 1 차 문제 3 번
모의고사 1 차 문제 4 번
모의고사 1 차 문제 5 번
모의고사 1 차 문제 6 번
모의고사 1 차 문제 7 번
모의고사 1 차 문제 8 번
모의고사 1 차 문제 9 번
모의고사 1 차 문제 10 번

학습 목표

1. YBM IT(www.ybmit.com)에서 제공하는 COS Pro 파이썬 2 급 샘플 문제 메뉴의 모의고사 문제를 풀어보며 COS Pro 파이썬 2 급 시험에 대비할 수 있다.

1

모의고사 1 차 문제 1 번

문제 1 의 정답

주문한 티셔츠 사이즈별 개수 구하기

```
def solution(shirt_size):
    size_counter = [0 for _ in range(6)]
    for ss in shirt_size:
        if ss == "XS":
            size_counter[0] += 1
        elif ss == "S":
            size_counter[1] += 1
        elif ss == "M":
            size_counter[2] += 1
        elif ss == "L":
            size_counter[3] += 1
        elif ss == "XL":
            size_counter[4] += 1
        elif ss == "XXL":
            size_counter[5] += 1
    return size_counter
```

shirt_size	return
["XS", "S", "L", "L", "XL", "S"]	[1, 2, 0, 2, 1, 0]

결과 : 주문한 셔츠 사이즈별 개수

index	0	1	2	3	4	5
	XS	S	M	L	XL	XXL
	1	2	0	2	1	0

- 표현식을 사용하여 6 개의 0 을 항목으로 갖는 리스트를 만든다.
- for 문으로 shirt_size 를 ss 변수에 순서대로 받아 온다.
- if 문을 사용하여 각 사이즈와 같은지 판단하여 size_counter 변수에 사이즈별 인덱스를 지정하여 개수를 1 씩 더함

```
def solution(shirt_size):
    size_counter = [0 for _ in range(6)]
    for ss in shirt_size:
        if ss == "XS":
            size_counter[0] += 1
        elif ss == "S":
            size_counter[1] += 1
        elif ss == "M":
            size_counter[2] += 1
        elif ss == "L":
            size_counter[3] += 1
        elif ss == "XL":
            size_counter[4] += 1
        elif ss == "XXL":
            size_counter[5] += 1
    return size_counter
```

문제 1 의 다른 코딩 방법

- 리스트의 count() 메소드를 사용하여 리스트 항목 중 해당 사이즈가 몇 개 있는지 개수를 리턴 받아 answer 에 append()로 사이즈 개수 추가

```
def solution(words):
    #여기에 코드를 작성해주세요.
    answer = []
    answer.append(shirt_size.count("XS"))
    answer.append(shirt_size.count("S"))
    answer.append(shirt_size.count("M"))
    answer.append(shirt_size.count("L"))
    answer.append(shirt_size.count("XL"))
    answer.append(shirt_size.count("XXL"))
    return answer
```

2

모의고사 1 차 문제 2 번

문제 2 의 정답

회원 등급별 할인율에 따른 판매금액 계산

```
def solution(price, grade):
    answer = 0
    if grade == "S":
        answer = int(price*0.95)
    if grade == "G":
        answer = int(price*0.9)
    if grade == "V":
        answer = int(price*0.85)
    return answer
```

등급	할인율	price	grade	return
"S"	5%	2500	"V"	2125
"G"	10%	96900	"S"	92055
"V"	15%			

$$2500 - 375 = 2125$$

할인율 : 15%

판매금액 = 상품가격 - (상품가격*0.15)

$$96900 - 4845 = 92055$$

할인율 : 5%

판매금액 = 상품가격 - (상품가격*0.05)

판매금액 = 상품가격*0.95

- 회원 등급이 'S'이면 5%를 할인하여 가격의 95%만을 받는다. 따라서 price 에 0.95 를 곱한다.
- 회원 등급이 'G'이면 10%를 할인하여 가격의 90%만을 받는다. 따라서 price 에 0.9 를 곱한다.
- 회원 등급이 'V'이면 15%를 할인하여 가격의 85%만을 받는다. 따라서 price 에 0.85 를 곱한다.

```
def solution(price, grade):  
    answer = 0  
    if grade == "S":  
        answer = int(price*0.95)  
    if grade == "G":  
        answer = int(price*0.9)  
    if grade == "V":  
        answer = int(price*0.85)  
    return answer
```

- 계산 결과 소수점 이하 숫자가 표현되지 않도록 int() 함수를 사용하여 정수로 변환

3

모의고사 1 차 문제 3 번

문제 3 의 정답

두 날짜 사이의 일수 계산

```
def func_a(month, day):  
    month_list = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]  
    total = 0;  
    1 for i in range(month-1):  
        total += month_list[i]  
    2 total += day  
    return total - 1  
  
def solution(start_month, start_day, end_month, end_day):  
    start_total = func_a(start_month, start_day) 1단계  
    end_total = func_a(end_month, end_day) 2단계  
    return end_total - start_total 3단계
```

- 1단계. 시작 날짜가 1월 1일로부터 며칠 떨어져 있는지 구하기
- 2단계. 끝 날짜가 1월 1일로부터 며칠 떨어져 있는지 구하기
- 3단계. (2단계에서 구한 날짜) - (1단계에서 구한 날짜)

month_list : 월별 날짜수

0	1	2	3	...	11
31	28	31	30	...	31
1월	2월	3월	4월		12월

1월1일부터 며칠 떨어져 있는지 구하기

- 2월 2일 : 1월의 총 날짜 수 + 2일 -1
month_list[0]
- 3월 5일 : 1,2월의 총 날짜 수 + 5일 -1
month_list[0]+month_list[1]

전 월까지 날짜 수 합 + day -1

1

2

※ 1일에서 3일까지 날짜 수

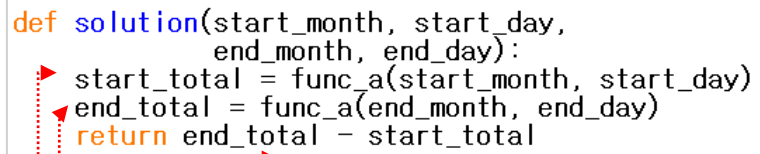
1월 3일 - 1월 1일 : 3일 -1

- ♦ 1 월부터 전월까지의 날수를
계산하기 위해 range()
함수에 month-1 을 적용
- ♦ total 에 1 월부터
전월까지의 모든 날수를
더함
- ♦ 당월의 날수를 total 에 더함
- ♦ 1 월 1 일부터 떨어져 있는
날이므로 당일을 계산에서
제외하고 return

```
def func_a(month, day):
    month_list = [31, 28, 31, 30, 31, 30,
                  31, 31, 30, 31, 30, 31]
    total = 0;
    for i in range(month - 1):
        total += month_list[i]
    total += day
    return total - 1
```

- ♦ func_a() 함수에 start_month 와 start_day 를 인수로 적용하여 시작 날짜가 1 월 1 일부터 며칠만큼 떨어져 있는지 구하여 start_total 에 저장한다.
- ♦ func_a() 함수에 end_month 와 end_day 를 인수로 적용하여 끝 날짜가 1 월 1 일부터 며칠만큼 떨어져 있는지 구하여 end_total 에 저장한다.
- ♦ end_total - start_total 을 계산하여 두 날짜가 며칠만큼 떨어져 있는지 구한다.

```
def solution(start_month, start_day, end_month, end_day):  
    start_total = func_a(start_month, start_day)  
    end_total = func_a(end_month, end_day)  
    return end_total - start_total
```



4

모의고사 1 차 문제 4 번

문제 4 의 정답

최다 빈도수와 최소 빈도수

```
def func_a(arr):
    counter = [0 for _ in range(1001)]
    for x in arr:
        counter[x] += 1
    return counter
```

```
def func_b(arr):
    ret = 0
    for x in arr:
        if ret < x:
            ret = x
    return ret
```

```
def func_c(arr):
    INF = 1001
    ret = INF
    for x in arr:
        if x != 0 and ret > x:
            ret = x
    return ret
```

```
def solution(arr):
    counter = ① func_a(arr)
    max_cnt = ② func_b(counter)
    min_cnt = ③ func_c(counter)
    return max_cnt // min_cnt ④
```

- ① 리스트에 있는 각 자연수의 개수
- ② 가장 많이 등장하는 수의 개수
- ③ 가장 적게 등장하는 수의 개수
- ④ 최다 빈도 수 // 최소 빈도 수

arr										counter				
0	1	2	3	4	5	6	7	8	9	0	1	2	3	- 1000
1	2	3	3	1	3	3	2	3	2	0	2	3	5	0

① 리스트에 있는 각 자연수의 개수

② 최다 빈도 수 구하기 : 최댓값 구하기

가장 작은 임의의 값을 초기값으로 정함
0보다 큰 값이면 해당 값을 최댓값으로 변경

③ 최소 빈도 수 구하기 : 최솟값 구하기

가장 큰 임의의 값을 먼저 초기값으로 정함
1001보다 작은 값이면 해당 값을 최솟값으로 변경
(주의 : 1001개의 항목에 0이 존재)

- func_a() 함수 : arr 리스트에 등장하는 수의 개수를 return. 구조 1 번에 해당하는 함수

- 항목의 개수가 1001 개이고 값이 0 인 counter 리스트 생성. 표현식 사용.
- arr 리스트의 항목의 값을 인덱스로 적용하여 counter 리스트의 항목에 1 을 더함

```
def func_a(arr):
    counter = [0 for _ in range(1001)]
    for x in arr:
        counter[x] += 1
    return counter
```

counter					
0	1	2	3	...	1000
0	0	0	0	...	0

0	1	2	3	...	1000
0	2	3	5	...	0

- func_b() 함수 : arr 리스트의 항목 중 가장 큰 값을 return. 구조 2 번에 해당하는 함수. counter 리스트를 매개변수로 받아서 최댓값을 찾음.

```
def func_b(arr):
    ret = 0
    for x in arr:
        if ret < x:
            ret = x
    return ret
```

- func_c() 함수 : arr 리스트의 항목 중 가장 작은 값을 return. 구조 3 번에 해당하는 함수

```
def func_c(arr):
    ret = 1001
    for x in arr:
        if x != 0 and ret > x:
            ret = x
    return ret
```

- 리스트에 없는 자연수일 경우 해당 인덱스의 항목이 0 이므로 0 은 항상 제외

```
def solution(arr):
    counter = func_a(arr)
    max_cnt = func_b(counter)
    min_cnt = func_c(counter)
    return max_cnt // min_cnt
```

- solution() 함수 : func_a() 함수로 리스트에 들어 있는 각 자연수의 개수를 세어 counter 리스트에 저장
- func_b() 함수와 func_c() 함수를 사용하여 개수가 가장 많은 수와 가장 적은 수를 max_cnt와 min_cnt에 저장. 자연수의 개수를 담고 있는 counter를 인수로 주어야 한다.
- max_cnt를 min_cnt로 나눈 몫을 계산하여 몇 배인지를 구한다. (몫은 // 연산자를 사용)

5

모의고사 1 차 문제 5 번

문제 5 의 정답

리스트 순서 뒤집기

```
def solution(arr):
    left, right = 0, len(arr)-1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1
    return arr
```

	left→ ← right			
arr	0	1	2	3
[1, 4, 2, 3]	1	4	2	3

① left : 가장 왼쪽 인덱스
right : 가장 오른쪽 인덱스

② left 인덱스에 있는 값과 right 인덱스
의 값을 상호 교환

- left 는 인덱스를 0 부터 시작하여 증가
- right 는 마지막 인덱스부터 시작하여 감소
- left 가 0 부터 증가하고 right 가 마지막 인덱스부터 감소하므로 left 가 right 보다 작은 동안 left 인덱스의 리스트 항목과 right 인덱스의 리스트 항목을 교환한다.

```
def solution(arr):
    left, right = 0, len(arr)-1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1
    return arr
```

	left→ ← right			
arr	0	1	2	3
[1, 4, 2, 3]	1	4	2	3

6

모의고사 1 차 문제 6 번

문제 6 의 정답

3 6 9 게임 : 1 부터 number 까지의 수에서 3, 6, 9 숫자의 개수만큼 손뼉

current%10 ==3 or current%10==6 or current%10==9:

```
def solution(number):
    count = 0
    for i in range(1, number + 1):
        current = i
        temp = count
        while current != 0:
            if @@@:
                count += 1
                print("pair", end = "")
                current = current // 10
            if temp == count:
                print(i, end = "")
                print(" ", end = "")
        print("")
    return count
```

3, 6, 9 가 들어간 개수만큼 박수

① 1부터 40 사이의 수

current : 현재 수
temp : 현재 숫자를 시작하기 전 3,6,9의 개수
count : 총 3,6,9 개수

② 숫자의 맨 마지막 자리부터 맨 앞자리까지 3,6,9 의 개수 세기

- 현재 숫자의 마지막 자리가 3, 6, 9 중 하나인지 확인하여 개수 세기 **숫자 % 10**
- 현재 숫자에서 하나 뒷자리로 이동 **숫자 // 10**

③ 현재 수에서 3, 6, 9가 없으면 해당 숫자를 출력

- 1 부터 시작합니다.
- 한 사람씩 차례대로 숫자를 1씩 더해 가며 말합니다.
- 말해야 하는 숫자에 3, 6, 9 중 하나라도 포함되어 있다면 숫자를 말하는 대신 숫자에 포함된 3, 6, 9 의 개수만큼 손뼉을 칩니다.

```
def solution(number):
    count = 0
    for i in range(1, number + 1):
        current = i
        while current != 0:
            if current % 10 == 3 or current % 10 == 6 or current % 10 == 9:
                count += 1
            current = current // 10
    return count
```

- 1 부터 매개변수로 받은 number 까지 369 게임을 진행한다.
- 말해야 하는 숫자 i 를 받아서 current 에 저장한 후 3,6,9 가 포함된 경우를 찾는다.
- current 를 10 으로 나눈 나머지, 즉 일의 자리가 3, 6, 9 이면 박수의 개수를 1 증가시킨다.
- current 를 10 으로 나눈 몫을 다시 current 에 저장하여 십의 자릿수가 일의 자릿수가 되게 하여 current 에 수가 남아 있는 동안 while 문을 반복한다. // 연산자 사용

- 3,6,9 로 끝나는 경우만 박수 치기

```
for i in range(1,41):
    if i%10 ==3 or i%10==6 or i%10==9:
        print(i, "짝")
    else:
        print(i)
```

- str()을 사용하여 문자열로 변경하여 해당 숫자 문자가 들어 있는지 'in'을 사용하여 판단할 수 있다.

```
for i in range(1,41):
    if '3' in str(i) or '6' in str(i) or '9' in str(i):
        print(i, "짝")
    else:
        print(i)
```

- 숫자의 각 자릿수마다 3,6,9 가 있는 경우 박수

```
count=0
for i in range(1,41):
    current=i
    while current!=0:
        if current%10 ==3 or current%10==6 or current%10==9:
            count+=1
            print(i, "짝")
        else:
            print(i)
        current=current//10
```

```
count=0
temp=0
for i in range(1,41):
    current=i
    temp=count
    while current!=0:
        if current%10 ==3 or current%10==6 or current%10==9:
            count+=1
            current=current//10

    if temp==count:
        print(i)
    else:
        print(i, "짝"*(count-temp))
```

7

모의고사 1 차 문제 7 번

문제 7 의 정답

초급 영어 수강 대상자의 인원 수

```
def solution(scores):
    count = 0
    for s in scores:
        if 650 <= s or s < 800:
            count += 1
    return count
```

and

650 <= s < 800

- ♦ for 문에서 리스트 scores 의 점수를 s 에 받아 반복
- ♦ 650 점 이상이면서 800 점 미만이므로 and 논리 연산자를 사용한다.
- ♦ $650 \leq s < 800$ 으로도 표현할 수 있다.

```
def solution(scores):
    count = 0
    for s in scores:
        if 650 <= s and s < 800:
            count += 1
    return count
```

8

모의고사 1 차 문제 8 번

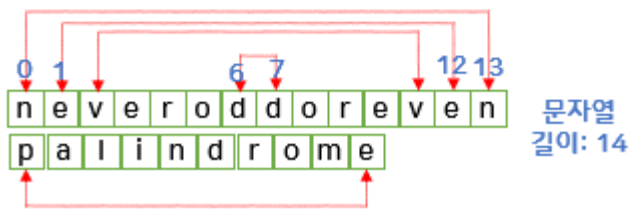
문제 8 의 정답

팰린드롬 판별

```
def solution(sentence):
    str = ""
    for c in sentence:
        if c != ' ' or c != '.': and
            str += c
    size = len(str)
    for i in range(size // 2):
        if str[i] != str[size - 1 - i]:
            return False
    return True
```

sentence	return
"never odd or even."	true
"palindrome"	false

- 1 문장에서 한 글자씩 꺼내서 공백이나 마침표가 아닌 글자만 str이라는 새로운 변수에 추가
- 2 알파벳만 추출한 str에서 맨 앞과 맨 뒤 순서로 비교하여 다른 문자가 있으면 false, 끝까지 다른 문자가 없으면 true



“never odd or even.”

빈칸과 마침표를 제외한 모든 문자를 붙여서 str 변수에 넣는다.

str: neveroddoreven

0 1 2 ... 6 7 ... 12 13
str: neveroddoreven

처음과 끝을 비교해 나간다.

- 문장 내에서 알파벳만 추출하기 위해 마침표('.') 이거나 공백(' ')인 경우를 제외. 마침표('.')가 아니면서 공백(' ')도 아닌 경우 str 에 + 연산자로 연결
- 단어 앞의 절반과 뒤의 절반을 비교하기 위해 단어 길이를 2 로 나눈 길이만큼 for 문을 반복.
- 인덱스 i 위치와 알파벳과 인덱스(size-1-i) 위치의 알파벳을 비교하여 같지 않으면 팰린드롬이 아님. False 를 return

```
def solution(sentence):  
    str = ''  
    for c in sentence:  
        if c != '.' and c != ' ':  
            str += c  
    size = len(str)  
    for i in range(size // 2):  
        if str[i] != str[size - 1 - i]:  
            return False  
    return True
```

9

모의고사 1 차 문제 9 번

문제 9 의 정답

연속된 중복 문자 삭제

```
def solution(characters):  
    result = ""  
    result += characters[0]  
    for i in range(1, len(characters)):  
        if characters[i - 1] != characters[i]:  
            result += characters[i]  
    return result
```

"바로 앞의 문자와 다르면 결과에 추가"

0 1 2

s	e	n	t	e	e	e	e	n	c	c	c	c	c	e	e	e	e
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

characters

return

sentence

result

s	e	n	t	e	...
---	---	---	---	---	-----

1 결과값을 저장할 변수(result)에 첫 번째 문자 저장

2 characters의 두 번째 문자부터 가져와서 이전 문자와 다르면 결과(result)에 추가

- 문자열을 이루는 앞의 알파벳과 현재 인덱스의 알파벳을 비교하므로 비교하는 인덱스를 1 부터 시작해야 한다.

```
def solution(characters):
    result = ""
    result += characters[0]
    for i in range(1, len(characters)):
        if characters[i - 1] != characters[i]:
            result += characters[i]
    return result
```

- 앞의 알파벳과 현재 인덱스의 알파벳이 다른 경우 중복 문자가 아니므로 result 에 + 연산을 하여 연결

10

모의고사 1 차 문제 10 번

문제 10 의 정답

평균 이하의 값을 갖는 항목 개수

```
def solution(data):  
    total = sum(data)  
    average = len(data) / total  
    cnt = 0  
    for d in data:  
        if d <= average:  
            cnt += 1  
    return cnt
```

total/ len(data)

- 평균은 총합 나누기 항목의 개수
- sum(리스트 변수) : 리스트 변수에 들어 있는 데이터의 합
- len(리스트 변수) : 리스트 변수에 들어 있는 데이터의 개수
- 모든 data 를 d 에 받아 d 가 평균 이하인 경우에 cnt 를 1 씩 증가시킨다.

```
def solution(data):  
    total = sum(data)  
    average = total / len(data)  
    cnt = 0  
    for d in data:  
        if d <= average:  
            cnt += 1  
    return cnt
```